Niagara 4 point deployment

Publishing and Consuming Points

Capability matrix across oBIX and REST API v2

Verified on Niagara 4.14.0.162 · server exports and client collection

Every result confirmed by an independent client off-station

Shanghai Gline Net Co., Ltd.

@jason912 · jason.zhang@gline-net.com

Two channels, two different jobs

Neither replaces the other each owns a step the other cannot perform

oBIX

HTTP + Basic Auth

Standard oBIX endpoint at /obix/config. The same server that publishes exports answers the queries.

Uniquely owns

- bind BOrd slots (pointOrd, point)
- write DynamicEnum (escaped name)
- history and alarm queries

REST API v2

HTTP + Basic + CIDR

The glineRestApi module. A batch of 11 operations in one POST, atomic per request.

Uniquely owns

- create / copy / rename / delete
- extensions, facets, link, invoke
- program source and compile

Rule of thumb: if a step touches a component's structure, REST does it. If it touches a typed slot the REST encoder cannot express, oBIX does it.

Capability matrix component structure

Yes / No / Partial, each cell measured against an independent client

Capability	oBIX	REST v2	Note
Read component tree and point values	Yes	Yes	same data, different shapes
Write a point value	Yes	Yes	oBIX must POST to /set/
Write an existing slot	Yes	Yes	oBIX PUT <value>
Create a component	No	Yes	oBIX: BadUriErr, every variant
Copy a component	No	Yes	REST: cp carries every slot
Rename a component	No	Yes	REST: new name goes in NEW
Delete a component	No	Yes	REST: remove
Link two components	No	Yes	REST: link
Invoke an action	No	Yes	action name in N
Add history extension	No	Yes	three ext types
Add alarm extension	No	Yes	alarm:AlarmSourceExt
Edit facets	No	Yes	v2.4 and later
BQL query	No	Yes	REST BQL returns rows

Capability matrix data, edge cases, gaps

The rows at the bottom are the ones that decide a design

Capability	oBIX	REST v2	Note
Read history records	Yes	No	oBIX has the query endpoints
Read alarm records	Yes	No	oBIX: ~alarmQuery
Transfer files (file:^)	No	Yes	REST is sandboxed
Write proxy in1 in16	No	Yes	oBIX set node is inert
Write proxy out	No	Yes	bypasses the priority chain
Write a BOrd slot	Yes	No	only oBIX PUT <uri> one channel
Write DynamicEnum	Yes	No	escaped name from ~range one channel
Clear a slot to true null	No	No	not offered by either channel
Write a NaN instead of null		Yes	stored as nan, still [overridden]

Two capabilities have exactly one channel each. A workflow that needs either of them cannot be built on a single channel plan for two, and test the boundary between them.

Publishing a station point outward four protocols, three steps

"Add a point" is never one operation; the step that fails is usually the second

Protocol	1 · Create the object	2 · Bind source / address	3 · Public identity	Channels
BACnet export	REST add bacnet:*Descriptor	pointOrd oBIX PUT <uri>	objectId REST mod	REST + oBIX
oBIX export	REST add obixDriver:ObixExport	point oBIX PUT <uri>	binding is the identity	REST + oBIX
Modbus TCP slave	REST cp (add leaves NullProxyExt)	dataAddress REST mod	binding is the identity	REST only
SNMP agent	REST add SnmpExportFolder first	objectIdentifier REST mod	OID is the identity	REST only

Why step 2 breaks

BOrd slots reject REST strings outright BString cannot be cast to BOrd. The XML API is the only channel that writes them.

Modbus slave points cannot be created with add: the result carries NullProxyExt and has no dataAddress slot at all.

A BACnet descriptor renames itself the moment objectId is written analogValue:100, not the name you typed.

The two traps that cost the most time

Trap 1 · dataType travels with the copy

cp copies every slot, including the register interpretation. An integer register read through Float Type decodes two registers as one IEEE-754 value: the point reads 0.00 with no fault and no hint.

```
out = 0.00 {stale}     faultCause = ""
readValue = 0.00 {ok}  enabled = true
```

Fix: set proxyExt/dataType = integerType over oBIX.

Trap 2 · the status facets lie

Four independent signals report success on a point that never receives data. None of them is a measurement of collection.

device {ok}	driver accepted the config
health = Ok	PingMonitor ran, separate path
readValue {ok}	a value object exists
readStatus okNotActive	"not active" is literal

Fix: read pollScheduler and change the value outside.

An empty faultCause is the normal appearance of a wrongly decoded point and of a point that is not polled at all. The two are distinguishable only by counters and by an external value change.

Is the client collecting? Two counters settle it

Read pollScheduler twice, ten seconds apart, before touching any configuration

Reading	Value	Conclusion
pollScheduler.totalPolls	increasing	scheduler running look downstream (decode, mapping)
pollScheduler.totalPolls	frozen	nothing is being polled at all
pollScheduler.normalCount	current = 1	a point is in the queue
pollScheduler.normalCount	current = 0	the queue is empty not "polling failed"

Same station, same minute

Network	Scheduler class	totalPolls	normalCount
SnmpNetwork1	NPollScheduler	122 over 17s	current = 1
ModbusTcpNetwork	BasicPollScheduler	0 over 0ms	current = 0

normalCount = 0 is not a failure of polling. It means no point was ever registered. Read both numbers before changing anything: they split the problem into "queue empty" and "poll runs but the value is wrong".

The only proof: change the value outside

Status facets, health monitors and copied values can all fake a healthy point

Modbus slave loop on one station

external master writes, client point on the same station follows

FC06 write HR[1] = 5555
point C1 = 5555.00 {ok}

FC06 write HR[1] = 6666
point C1 = 6666.00 {ok}

FC06 write HR[1] = 4321
point C1 = 4321.00 {ok}

totalPolls climbing throughout
slave currentConnections = 1
three distinct values, each reflected

What this rules out in one step

A duplicated device that shows data proves nothing copied points carry the source's last value, and a clone looks healthy while collecting nothing.

A working neighbour exonerates the network, the driver module and the protocol at once: same second, same network, one device collecting and one not.

Only a value change at the far end separates "polled and decoded" from "looks plausible".

Choosing a channel by task

Ordered so the first channel named is the one to reach for

Task	First choice	Why	Fallback
Create or tear down components	REST v2	only channel with add / remove	
Build points in bulk from a template	REST v2	one POST, atomic batch	
Bind a source point to an export	oBIX	only channel that writes BOrd	
Set a NIC / IP / adapter dropdown	oBIX	only channel that writes DynamicEnum	
Read trend or alarm data	oBIX	query endpoints exist	
Ad-hoc table query	REST v2 BQL	returns real rows	
Transfer files	REST v2	sandboxed file:^	
Diagnose collection	REST v2 (read-only)	scheduler + poll config, no side effects	

Summary

What holds, and what stays open

Established	oBIX only	REST v2 only	Open
Read and write of point values	BOrd slot binding	Component lifecycle	Three Modbus client devices
History and alarm queries	DynamicEnum writes	Extensions and facets	Non-zero consecutivePointsToPoll
Outside-client verification	True-null clearing is unavailable	Bulk atomic batches	BACnet client coverage

Two rules worth carrying into any deployment

Verify with an independent client, from another host, using a value you changed yourself. Everything inside the station can report success.

When one device collects and an identical one beside it does not, stop testing the protocol the difference is in the objects, not the driver.

Shanghai Gline Net Co., Ltd. · @jason912 · jason.zhang@gline-net.com

Companion documents: niagara-bacnet-server-points.md, niagara-modbus-server-points.md, niagara-obix-server-points.md, niagara-client-collection.md